



Qiita 実装仕様・新人向け

ライブラリゼロで作る「研修ツール」

単一 HTML ファイルに全部入れる実装入門

JavaScript / HTML / 社内ツール / フロントエンド

「面白きこともなき世を面白く」



01 なぜ単一 HTML？ - 現場の制約から

●研修会場の現実

- 会場 PC にソフトをインストールできない
- ネットワークが不安定、または制限されている
- 「USB で渡してすぐ動く」が正義

●導かれる要件

- 依存ゼロ・ビルドなし・オフライン動作
- 答え： index.html 1 枚構成（ダブルクリックで起動）



02 土台の 5 パターン（全体像）

1. タブ UI

クラスの付け替えだけ

2. 状態管理

1 つのオブジェクトに集約

3. 自動保存

localStorage + try/catch

4. 直接編集

contenteditable + 委譲

5. ファイル出力

Blob + a 要素 + 日時刻印

この 5 点セットで、インストール不要の現場ツールが成立する



03 パターン1：タブ UI はクラス付け替えだけ

- data-tab 属性でボタンとセクションを紐づけ
- CSS は 2 行: .tab{display:none} / .tab.active{display:block}
- ライブラリ不要・履歴も状態も持たない最小構成

```
function go(id){  
  document.querySelectorAll('.tab')  
    .forEach(t => t.classList.toggle(  
    'active', t.id === id));  
  document.querySelectorAll('#nav button')  
    .forEach(b => b.classList.toggle(  
    'active', b.dataset.tab === id));  
}
```

```
<button data-tab="tab-search">  
  検索 </button>  
<section id="tab-search"  
  class="tab">...</section>
```



04 パターン2：状態は1つのオブジェクトに

- データの置き場所を S ひとつに統一
- 保存も復元も JSON.stringify / parse の一撃
- フレームワーク利用時にも効く考え方

```
const emptyState = () => ({
  meta: {
    title: '',
    world: 'うさうさラーメン店'
  },
  questions: [], // 質問 DB
  qlog: []       // 当日の質問ログ
});
let S = emptyState();

// 保存
JSON.stringify(S);
// 復元
S = Object.assign(
  emptyState(), JSON.parse(saved));
```



05 パターン3：自動保存は try/catch で包む

- localStorage は環境により例外を投げる
 - (プライベートモード・企業ポリシー等)
- 保存に失敗してもツール本体は動き続けること
- 逃げ道：JSON ファイルへの手動バックアップを併設

```
function autosave(){
  try {
    localStorage.setItem(
      'usausa_qgen_v1',
      JSON.stringify(S));
  } catch(e) {
    /* 保存不可でも落とさない */
  }
}

// 編集中は 600ms デバウンス
let timer;
function autosaveSoon(){
  clearTimeout(timer);
  timer = setTimeout(autosave, 600);
}
```



06 パターン 4：表の直接編集は contenteditable

- 属性 1 つで Excel 風のセル編集が手に入る
- イベントは行ごとに付けず、表に 1 個だけ（委譲）
 - 行の追加・削除でも付け直し不要
- data-i / data-c で「どの行のどの列か」を運ぶ

```
<td contenteditable="true"
      data-i="0" data-c="question">
  VPC って何ですか? </td>

table.addEventListener('input', e => {
  const td = e.target;
  S.questions[td.dataset.i]
    [td.dataset.c]
    = td.innerText;
  autosaveSoon();
});
```



07 パターン5：ダウンロードは Blob + a 要素

- サーバーなしでブラウザだけでファイルを生成
- 日時プレフィックスを **download** 関数の中で必ず付与
 - 「どれが最新？」問題が消える、現場で一番効く仕様

```
function download(name, content, mime){
  name = stamp() + '_' + name;
  // 例：20260606_1530_想定質問集.xlsx
  const blob =
    new Blob([content], {type: mime});
  const a =
    document.createElement('a');
  a.href = URL.createObjectURL(blob);
  a.download = name;
  a.click();
  URL.revokeObjectURL(a.href);
}
```



08 新人さんが必ず踏む穴：エスケープ（XSS）

- 外部データを **innerHTML** に入れる前に必ずエスケープ
- インポート機能がある時点で「外部データ」は入ってくる
- 「自分しか使わないから」は通用しない
- 表示系の全箇所で `esc()` を通すのを習慣に

```
function esc(s){
    return String(s ?? '')
        .replace(/&/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;');
}

// 必ず esc を通してから差し込む
el.innerHTML =
    '<div class="q">'
    + esc(q.question) + '</div>';
```



09 まとめ

一行まとめ

タブ=クラス付け替え、状態= 1 オブジェクト、保存= `localStorage+JSON`、編集= `contenteditable`、出力= `Blob`。この 5 点セットで、インストール不要の現場ツールは作れる。

次回（中級編）：ライブラリなしで `.xlsx` を読み書きする実装 - 「エラー :509」事件の顛末

「面白きこともなき世を面白く」

